Counterfactual Data-Fusion for Online Reinforcement Learners: An Evaluation

Adethya Srinivasan

May 2026

1 Introduction and Context

1.1 The Context of Online Learning

Sequential decision-making under uncertainty is a foundational problem in artificial intelligence. The Multi-Armed Bandit (MAB) problem is one of the most studied models for this class of problems [3]. In the classic MAB setting, an agent repeatedly selects from a finite set of K actions (or "arms") $\mathcal{X} = \{x_1, \ldots, x_K\}$ and receives a stochastic reward $Y \in \mathbb{R}$ after each selection. The objective is maximising cumulative expected reward over T time steps, or, minimising cumulative *regret* - deficit in reward relative to an oracle that always selects the optimal arm.

In all its applications, the agent must solve the *exploration-exploitation trade-off*: should it continue exploiting the arm it currently believes to be best, or explore undersampled arms to reduce uncertainty about their true quality?

Classical bandit algorithms address this trade-off when the environment is *exchangeable* - choice of arm is statistically independent of underlying world state. Under this assumption, the reward distribution $P(Y \mid X = x)$ observed in practice estimates true arm quality x.

1.2 The Problem of Unobserved Confounding

The challenge addressed by Forney, Pearl, and Bareinboim (2017) [2] arises when this exchangeability assumption fails - a failure that is endemic in real-world deployments. Consider a hidden variable U that influences both the agent's action X and resulting

reward Y [1] - an *unobserved confounder (UC)*. This creates a wedge between two fundamentally different quantities: the *observational* conditional $P(Y \mid X = x)$ (natural co-occurrences of arms and rewards) and the *experimental/interventional* distribution $P(Y \mid do(X = x))$ (reward an agent would receive if it were to physically force the selection of arm x independently of U).

This distinction has severe practical consequences. A standard ε-greedy agent, for instance, updates a Q-table indexed by arm identity. When a UC is present, the rewards that arm x delivers depends on hidden state U. The agent's estimates, therefore, converge to a *confounded* mixture of state-conditional payoffs, rather than the causal effect of the action. The agent may systematically identify a suboptimal arm as best, because the high-reward states are the states in which the confounder steers it away from that arm. As the authors demonstrate via the "Greedy Casino" experiment, this effect can be extreme: an adversarial environment can be engineered so that $P(Y = 1 \mid X = x) = 0.20$ for every arm x while the true causal reward $P(Y = 1 \mid do(X = x)) = 0.40$ for all arms.

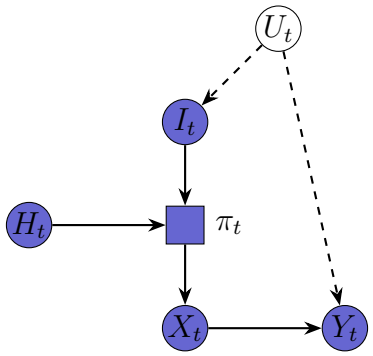
1.3 Report Objective

We provide an evaluation of Forney et al. (2017) [2] through two complementary lenses. First, we dissect the paper's causal machinery, examining how Structural Causal Models (SCMs), *do*-calculus, and counterfactuals are mobilised to construct a principled bandit algorithm - the *Multi-Armed Bandit with Unobserved Confounders* (MABUC). Second, we present and analyse computational simulations that reproduce the paper's core experimental claims. Three agents are benchmarked on the Greedy Casino environment: a standard ε-greedy MAB that ignores intent, an RDC-based agent that conditions decisions on intent, and a Data-Fusion RDC agent that exploits cross-intent and cross-arm algebraic identities to accelerate learning. Results are reported for both a fixed-ε regime and ε-decay regime, and we discuss what the results reveal about the speed, stability, and structural preconditions of counterfactual bandit learning.

2 Causal Inference Methodology

2.1 The Causal Graph and Structural Causal Model

The Greedy Casino setup can be illustrated using the following Directed Acyclic Graph (DAG):



For the MABUC problem at time step t, the endogenous variables are:

- I_t - the agent's **Intent**: the arm it would naturally select without deliberate intervention;
- X_t - the agent's **Action**: the arm it actually selects after applying its policy;
- Y_t - the **Reward**: the outcome received after pulling X_t.

The exogenous variable U_t encodes the *unobserved state of the world* at time t - for example, the simultaneous effect of a gambler's sobriety and the blinking state of the slot machines in the Greedy Casino (see A). The structural equations governing the model are:

$$I_t \leftarrow f_I(U_t), \qquad X_t \leftarrow \pi_t(I_t, H_t), \qquad Y_t \leftarrow f_Y(X_t, U_t)$$

where π_t denotes the agent's policy at time t and H_t is the agent's accumulated history of past intents, actions, and rewards.

There are only two arrows from U_t ($U_t \rightarrow I_t$, $U_t \rightarrow Y_t$) resulting in a *back-door path* $X \leftarrow I \leftarrow U \rightarrow Y$ which, if not adjusted for, renders any purely observational estimate of arm quality causally invalid.

2.2 Observational vs. Experimental Data

The DAG above makes precise the distinction between two datasets that the paper exploits. In the *observational world*, agents behave naturally: the hidden state U_t draws an intent I_t, and the agent acts on that intent. The resulting data generates the conditional distribution:

$$P(Y \mid X = x) \;=\; \sum_{u} P(Y \mid X = x, U = u)\, P(U = u \mid X = x)$$

Because U is a common cause of X and Y, the conditioning event $X = x$ selects a *non-representative* sub-population with respect to U - precisely those hidden states that tend to produce the choice of arm x. In the Greedy Casino, this means that whenever $X = x$ is observed, the agent is almost certainly in the hidden state that the casino has rigged. Consequently, $\mathbb{E}[Y \mid X = x] = 0.20$ for all arms, despite no arm being intrinsically poor.

The *experimental world*, by contrast, is accessed by applying Pearl's *do*-operator. The intervention $do(X = x)$ severs incoming arrows to X in the DAG. This eliminates the back-door path and yields:

$$P(Y \mid do(X = x)) \;=\; \sum_{u} P(Y \mid X = x, U = u)\, P(U = u)$$

Under a uniform hidden-state distribution $P(U = u) = 0.25$, this correctly estimates $\mathbb{E}[Y \mid do(X = x)] = 0.40$ for all arms. The confounding bias is:

$$\text{Bias}(x) \;=\; P(Y \mid do(X = x)) - P(Y \mid X = x)$$

which equals $+0.20$ in the Greedy Casino and is non-zero for any environment where $P(Y \mid do(X)) \neq P(Y \mid X)$.

2.3 Counterfactual Data Fusion and the Regret Decision Criterion

Experimental data from a randomised trial yields the correct *average* causal effect, but it is still insufficient for an online learning agent that experiences a specific hidden state at every time step. What the agent truly needs is a *state-conditional* estimate: given that I am currently in the hidden state that produces intent $I_t = i$, what is the expected reward if I pull arm a?

This quantity is a **counterfactual** expectation, written $\mathbb{E}[Y_{X=a} \mid X = i]$, where $Y_{X=a}$ denotes the potential outcome under the hypothetical intervention $do(X = a)$,

and the conditioning event $X = i$ anchors the query to the specific hidden state U that generated intent i. Counterfactuals of this type belong to the third rung of Pearl's "ladder of causation" [4] and, generally, are not identifiable from observational or experimental data alone. Forney et al. (2017) [2] establish the key identifiability result: because the agent explicitly records its intent I_t before acting, any off-intent action taken constitutes a counterfactual observation, and the full matrix of state-conditional payoffs becomes estimable from the agent's own trajectory.

The agent's decision rule - the **Regret Decision Criterion (RDC)** - follows directly:

$$X_t^* \;=\; \arg\max_{a \in \mathcal{X}} \; \mathbb{E}[Y_{X=a} \mid X = I_t]$$

The RDC instructs the agent to: (1) observe its current intent I_t, which serves as a proxy for the hidden U_t; (2) compute the counterfactual expected reward for each arm a, conditioned on the current intent state; and (3) select the arm that maximises this expectation. In the Greedy Casino, when $I_t = 3$ (intent to play Machine 3, characteristic of the drunk-and-blinking state), the state-conditional payoffs from the payout matrix are $\{0.60, 0.50, 0.30, 0.20\}$ for arms $\{0, 1, 2, 3\}$ respectively. The arg max resolves to arm 0, yielding a 60% win rate in place of the 20% a naive agent would receive.

The bridge between the available datasets and the full counterfactual matrix is provided by the law of total expectation applied to the experimental distribution:

$$\mathbb{E}[Y_{x_r}] \;=\; \sum_{k=1}^{K} \mathbb{E}[Y_{x_r} \mid x_k]\, P(x_k)$$

The left-hand side is known from experimental data. The diagonal terms $\mathbb{E}[Y_{x_r} \mid x_r]$ reduce to observational quantities by the *consistency axiom* [4] (if intent equals action, observational and counterfactual coincide). The off-diagonal terms $\mathbb{E}[Y_{x_r} \mid x_k]$, $k \neq r$, are the genuinely counterfactual cells the agent must estimate through online exploration or algebraic inference.

2.4 The MABUC Algorithm and Data Fusion Strategies

The authors operationalise the RDC via three complementary estimation strategies that fuse the available datasets to fill the counterfactual matrix faster than pure exploration alone. Full algebraic derivations of each strategy are provided in B.

Cross-Intent Learning B.2 isolates any target cell by rearranging the master equation (B.1), subtracting the contribution of all known intents from the experimental total:

$$\mathbb{E}_{\text{XInt}}[Y_{x_r} \mid x_w] \;=\; \frac{\mathbb{E}[Y_{x_r}] - \sum_{i \neq w} \mathbb{E}[Y_{x_r} \mid x_i]\, P(x_i)}{P(x_w)}$$

Cross-Arm Learning B.3 exploits the fact that the intent prior $P(x_w)$ is a shared constant across arms. By equating the expressions for two different arms x_r and x_s under the same intent x_w and solving for the unknown cell, the agent can estimate the payoff of arm r under intent w from collected data for arm s under that same intent. When multiple reference arms are available, estimates are aggregated using an inverse-variance weighting:

$$\mathbb{E}_{\text{XArm}}[Y_{x_r} \mid x_w] \;=\; \frac{\sum_{s \neq r} \hat{h}(x_r, x_w, x_s) \,/\, \sigma^2_{x_s, x_w}}{\sum_{s \neq r} 1 \,/\, \sigma^2_{x_s, x_w}}$$

Combined Estimator B.4 performs a final inverse-variance weighted fusion of all three sources - direct sampling, cross-intent inference, and cross-arm inference - into a single estimate:

$$\mathbb{E}_{\text{combo}}[Y_{x_r} \mid x_w] \;=\; \frac{\mathbb{E}_{\text{samp}}/\sigma^2_{\text{samp}} \;+\; \mathbb{E}_{\text{XInt}}/\sigma^2_{\text{XInt}} \;+\; \mathbb{E}_{\text{XArm}}/\sigma^2_{\text{XArm}}}{1/\sigma^2_{\text{samp}} \;+\; 1/\sigma^2_{\text{XInt}} \;+\; 1/\sigma^2_{\text{XArm}}}$$

In the implementation, the agent's history H_t is represented as a pair of matrices: a Q-table $Q(i, a) \approx \mathbb{E}[Y_{X=a} \mid X = i]$ and a count table $N(i, a)$ recording the number of times action a has been taken under intent i. At each step $\mathbb{E}_{\text{combo}}$ is computed for every arm a given the current intent i; the RDC then selects the arm with the highest fused estimate, subject to ε-greedy exploration to ensure continued coverage of the counterfactual matrix.

3 Computational Simulation and Reproduction

All code can be found here.

3.1 Simulation Setup

The simulation environment implements the Greedy Casino described in 1.2 (full source in C.1). The full state-conditional payout table and the observational/experimental paradox table are reproduced in A for reference. The environment exposes a $K = 4$

arm bandit with a 4×4 payout matrix $\mathbf{P}$, where entry $\mathbf{P}[a, u]$ gives the Bernoulli win probability for arm a under hidden state u:

$$\mathbf{P} = \begin{pmatrix} 0.20 & 0.30 & 0.50 & 0.60 \\ 0.60 & 0.20 & 0.30 & 0.50 \\ 0.50 & 0.60 & 0.20 & 0.30 \\ 0.30 & 0.50 & 0.60 & 0.20 \end{pmatrix}$$

The diagonal entries - those where the arm index equals the hidden-state index - are always 0.20, encoding the casino's adversarial trap: the arm the agent is naturally drawn to pays the minimum rate. Off-diagonal entries increase monotonically as the action diverges from the natural choice, reaching 0.60 when the agent fully defies its intent.

At each step, two independent binary variables $B_t, D_t \sim \text{Bernoulli}(0.5)$ are combined as $U_t = B_t + 2D_t$, yielding four equally probable hidden states with $P(U_t = u) = 0.25$. Since the gambler's natural choice is $I_t = U_t$, intent is the sole observable signal through which U_t manifests. U_t is never disclosed to any agent.

3.2 Algorithms Implemented

Three agents were implemented and benchmarked in parallel over $T = 50{,}000$ steps, sharing the same random seed for fair comparison.

Standard MAB (C.2) serves as the confounded baseline. It implements an ε-greedy algorithm with a fixed $\varepsilon = 0.1$ and one-dimensional Q-table $Q[a]$ indexed solely by arm identity. Critically, it receives no intent signal whatsoever; its choice ignores the current hidden state entirely. Its update rule is the standard incremental mean:

$$Q[a] \leftarrow Q[a] + \frac{1}{N[a]}\big(r - Q[a]\big)$$

This agent is designed to converge to the observational distribution $\mathbb{E}[Y \mid X = x]$, illustrating the confounding failure mode described in 1.2.

RDC Agent (C.3) implements the Regret Decision Criterion. It maintains a two-dimensional Q-table $Q[i, a]$ and count table $N[i, a]$, both indexed by intent i and action a. At decision time, it conditions on the current intent I_t and selects $\arg\max_a Q[I_t, a]$, subject to ε-greedy exploration. The update rule is identical to the

Standard MAB but applied to the intent-conditional cell $Q[I_t, a_t]$. The implementation also supports an ε-decay schedule - $\varepsilon \leftarrow \max(\varepsilon \cdot \delta, \varepsilon_{\min})$ applied after every step, with defaults $\varepsilon_0 = 0.1$, $\delta = 0.9995$, $\varepsilon_{\min} = 0.01$ - which was toggled between experimental runs to produce the two result figures discussed in 4.

Data-Fusion RDC Agent (C.4) extends the RDC Agent by overriding the action-selection step. Rather than consulting the raw Q-table, it computes the combined inverse-variance estimator $\mathbb{E}_{\text{combo}}[Y_{x_a} \mid x_i]$ for each arm. This agent is preloaded with the prior observational data ($\mathbb{E}[Y \mid X = x] = 0.20$ for all x) and experimental data ($\mathbb{E}[Y_x] = 0.40$ for all x) derived from the payout matrix. The intent prior is initialised to a uniform distribution, $P(I = i) = 0.25$, consistent with the equal probability of each hidden state.

3.3 Data Generation and Experimental Protocol

The simulation follows a shared-environment protocol to ensure that all three agents face statistically identical conditions at every step. At the start of each time step t, the shared hidden state U_t is fetched and the resulting intent I_t is broadcasted to the two intent-aware agents. Each agent then independently selects an action, after which a single reward is sampled per unique arm pulled, ensuring that any two agents selecting the same arm in the same step receive identical outcomes. The reward is drawn from a Bernoulli distribution: $Y_t \sim \text{Bernoulli}(\mathbf{P}[a_t, U_t])$.

Performance is tracked as the **cumulative average reward** at each step:

$$\bar{R}_t \;=\; \frac{1}{t}\sum_{\tau=1}^{t} Y_\tau$$

This metric converges to $\mathbb{E}[Y_{X=a^*} \mid X = I]$ for a fully converged intent-conditional policy, and to $\mathbb{E}[Y \mid X = a^*]$ for the Standard MAB. Two experimental configurations were run: one with fixed $\varepsilon = 0.1$ throughout (fig. 1), and one with the ε-decay schedule (fig. 2). All experiments used a fixed random seed of 42 for reproducibility.

The key assumption underlying the entire simulation is that the intent prior $P(I)$ is known and stationary. In practice, this means the agent must either be informed of the prior or estimate it from frequency counts. A secondary assumption is that the structural form $I_t = U_t$ holds deterministically - that is, the hidden state perfectly determines intent with no residual noise. We return to the implications of relaxing

these assumptions in Section 4.3.

4 Analysis and Evaluation

4.1 Results of the Simulation

The two figures below present the cumulative average reward $\bar{R}_t$ for all three agents over $T = 50{,}000$ steps, under two exploration regimes.

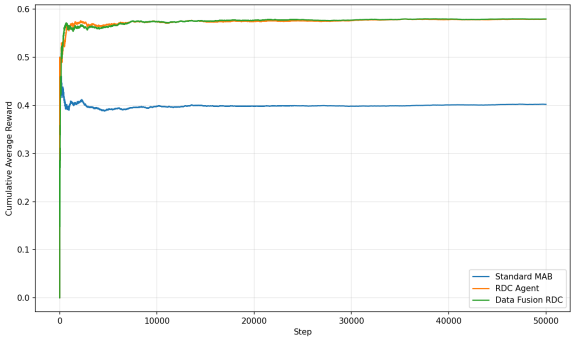


Figure 1: Results with a fixed $\varepsilon = 0.1$ throughout

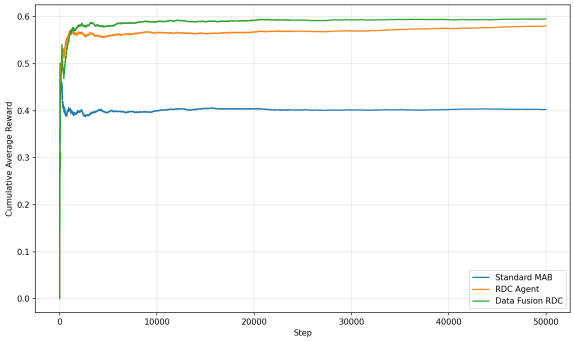


Figure 2: Results with the ε-decay schedule ($\varepsilon_0 = 0.1$, $\delta = 0.9995$, $\varepsilon_{\min} = 0.01$).

Across both figures, three broad patterns emerge. First, the Standard MAB (blue) stabilises rapidly at $\bar{R} \approx 0.40$ and flatlines thereafter. Second, both causal agents - the RDC Agent (orange) and the Data-Fusion RDC (green) - outperform the baseline, converging to reward levels in the range $[0.57, 0.60]$. Third, the Data-Fusion RDC consistently reaches its asymptotic performance level *earlier* than the plain

RDC Agent, though the eventual gap depends on the exploration regime.

A quantitative ceiling can be derived directly from the payout matrix. In the Greedy Casino, the optimal intent-conditioned policy always selects the arm diagonally opposite to its intent - the column maximum for each intent state always equals 0.60. The theoretical maximum for a perfect oracle is therefore $\mathbb{E}[Y^*] = 0.60$. Under fixed $\varepsilon = 0.10$, each agent devotes 10% of steps to uniformly random exploration, which yields an expected exploratory reward of $\mathbb{E}[Y \mid \text{random}] = 0.40$ (the experimental average under uniform intent). The expected long-run reward of a fully converged intent-conditioned policy is therefore:

$$\bar{R}^{\varepsilon}_{\infty} \;=\; (1-\varepsilon)\cdot 0.60 + \varepsilon \cdot 0.40 \;=\; 0.9 \times 0.60 + 0.1 \times 0.40 \;=\; 0.58$$

Both causal agents converge to $\approx$ 0.57-0.58 in fig. 1, in agreement with this analytical prediction. Under ε-decay, ε eventually falls to 0.01, shifting the ceiling to $(0.99)(0.60) + (0.01)(0.40) \approx 0.598$, which matches the asymptotic values of $\approx$ 0.585-0.59 observed in fig. 2.

The most significant difference between the two figures lies in the convergence dynamics rather than the asymptotic level. In fig. 1, the Data-Fusion RDC (green) pulls ahead of the RDC Agent (orange) within the first few hundred steps and briefly maintains a margin over the RDC Agent, up to approximately $t = 20{,}000$. In fig. 2, the two causal agents' trajectories remain distinguishable after $t \approx 50{,}000$: the shrinking exploration rate illustrates the advantage of the Data-Fusion strategy. This confirms that the paper's claim - data fusion accelerates learning - holds most strongly in regimes where the agent sustains meaningful exploration over a long horizon.

4.2 Why Did It Work? A Causal Perspective

The performance gap between the Standard MAB and the causal agents is not a matter of degree; it reflects a *categorical* difference in what each algorithm is learning.

The Standard MAB learns $Q[a] \to \mathbb{E}[Y \mid X = a]$. The casino's cyclic payout symmetry ensures this quantity equals 0.40 for every arm under a uniform hidden-state distribution - the arm-marginal reward is flat, presenting no gradient to ascend. No improvement beyond 0.40 is achievable without conditioning on intent.

The RDC Agent learns the function $Q[i, a] \to \mathbb{E}[Y_{X=a} \mid X = i]$. This is the intent-conditional payout matrix $\mathbf{P}$, the rows of which are *not* flat. When the intent

$I_t = 0$ is observed (sober, lights off), the Q-table row for intent 0 will, after sufficient exploration, contain $[0.20, 0.60, 0.50, 0.30]$, and the arg max correctly identifies arm 1 as optimal for that hidden state. The same logic applies to each of the four intent profiles, and the agent's policy effectively inverts the casino's trap at every step.

The Data-Fusion RDC achieves the same asymptotic performance but converges faster by supplementing direct samples with algebraic estimates. Early in learning, when $N[i, a]$ counts are small, the cross-intent estimator provides values for unvisited cells by redistributing the known experimental average; the cross-arm estimator triangulates from adjacent arms under the same intent. Inverse-variance weighting automatically favours the more stable source. The practical effect of this is visible in both figures as a shorter initial trough.

4.3 Limitations and Critique

Despite the compelling results, several important limitations constrain the applicability of the framework:

Identifiability requires perfect intent observation. The entire theoretical apparatus holds only if I_t is observed before the agent commits to an action. When intent is latent - user preferences inferred rather than declared, or $I = f(U)$ only approximately satisfied - the back-door path is no longer fully blocked, and counterfactual estimates are biased. Our simulation hardcodes $I_t = U_t$, the most favourable possible case.

Prior misspecification degrades fusion quality. The cross-intent estimator is sensitive to $P(x_w)$, here initialised to the true value of 0.25. A non-uniform or time-varying prior introduces systematic bias that inverse-variance weighting then propagates at high confidence - a failure mode absent from our controlled simulation.

The structured symmetry of the Greedy Casino is unusually favourable. The payout matrix's cyclic design places a 0.60 peak in every intent state, meaning the correct action is always uniquely determined and maximally rewarding. Real-world confounders are rarely this strong: as $\text{Bias}(x) \to 0$, the causal agents' advantage vanishes. When the confounder has negligible influence, observational and interventional distributions coincide and intent carries no information.

Scalability of the counterfactual matrix. The Q-table has K^2 entries. For the original $K = 4$ casino, this is modest. For generalised environments with large K,

the number of cells grows quadratically and the number of required exploratory transitions to achieve low variance across the full matrix grows proportionally. The data fusion strategies partially mitigate this via cross-arm inference, but the fundamental curse of dimensionality is not overcome, particularly in the cross-arm estimator whose variance propagation involves a sum over $K - 1$ reference arms.

5 Conclusion

Forney et al. (2017) [2] make a compelling case that the failure of classical bandit algorithms in confounded environments is not just a statistical problem requiring more data, but a *structural* problem requiring a change in the quantities being estimated. Standard ε-greedy agents converge to observational or interventional averages that are, by construction, insensitive to the hidden state the agent currently occupies. The paper's key insight is that the agent's intent I_t constitutes an observable proxy for the unobserved confounder U_t, and that this proxy unlocks the identifiability of state-conditional counterfactual expectations.

Our simulations confirm this claim. The intent-conditioned agents achieved cumulative average rewards of ≈ 0.58 under fixed exploration and ≈ 0.59 under ε-decay, against the Standard MAB's ceiling of ≈ 0.40 - a gain of approximately 47% in reward, closely matching the analytical ceiling derived from the payout structure. The data-fusion variant offered the clearest advantage during the early learning phase under fixed ε, validating the paper's algebraic acceleration mechanism. However, our analysis also identifies important boundary conditions: the framework requires deterministic intent observation, a well-specified intent prior, and a confounder whose effect on payouts is strong enough to generate an exploitable gradient. Within these conditions, causal data fusion represents a principled and demonstrably effective solution to a problem that pure statistical learning cannot address.

References

[1] Bareinboim, E., Forney, A., and Pearl, J. Bandits with unobserved confounders: A causal approach. In *Advances in Neural Information Processing Systems* (2015), C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28, Curran Associates, Inc.

[2] Forney, A., Pearl, J., and Bareinboim, E. Counterfactual data-fusion for online reinforcement learners. In *Proceedings of the 34th International Conference on Machine Learning* (06–11 Aug 2017), D. Precup and Y. W. Teh, Eds., vol. 70 of *Proceedings of Machine Learning Research*, PMLR, pp. 1156–1164.

[3] Lattimore, T., and Szepesvári, C. *Bandit Algorithms*. Cambridge University Press, 2020.

[4] Neuberg, L. G. Causality: Models, reasoning, and inference, by judea pearl, cambridge university press, 2000. *Econometric Theory 19*, 4 (2003), 675–685.

A Greedy Casino - Full Payout and Paradox Tables

A.1 State-Conditional Payout Matrix $\mathbb{E}[Y \mid X, B, D]$

The full 4×4 payout matrix used in the simulation. Rows index the arm pulled (X); columns index the hidden state (B, D). Asterisked entries mark the gambler's *natural* choice for each hidden state-always paid at 0.20.

Arm X	$D = 0, B = 0$	$D = 0, B = 1$	$D = 1, B = 0$	$D = 1, B = 1$
$X = 0$	0.20	0.30	0.50	0.60
$X = 1$	0.60	0.20	0.30	0.50
$X = 2$	0.50	0.60	0.20	0.30
$X = 3$	0.30	0.50	0.60	0.20

where X is the slot machine chosen ($X \in \{0, 1, 2, 3\}$ for 4 machines), D is the gambler's sobriety (the gambler is drunk (1) or not (0) and B is whether the machines are blinking (blinking (1) or not (0)).

Hidden state index: $U = B + 2D$*, so column 0 is* $(D = 0, B = 0)$*, column 1 is* $(D = 0, B = 1)$*, column 2 is* $(D = 1, B = 0)$*, column 3 is* $(D = 1, B = 1)$*.*

A.2 The Confounding Paradox: Observational vs. Experimental Expected Rewards

Arm X	$\mathbb{E}[Y \mid X]$ **(observational)**	$\mathbb{E}[Y \mid do(X)]$ **(experimental)**
$X = 0$	0.20	0.40
$X = 1$	0.20	0.40
$X = 2$	0.20	0.40
$X = 3$	0.20	0.40

The observational column reflects the casino's trap: natural gamblers are always steered into the state where their chosen arm pays 0.20. The experimental column reflects the randomised-trial average: $0.25 \times (0.20 + 0.30 + 0.50 + 0.60) = 0.40$ for every arm, by the payout matrix's row-sum symmetry.

B Derivation of the Data-Fusion Estimators

B.1 Master Equation

Starting from the law of total expectation, conditioning on intent $X = x_k$:

$$\mathbb{E}[Y_{x_r}] = \sum_{k=1}^{K} \mathbb{E}[Y_{x_r} \mid X = x_k]\, P(X = x_k)$$

The left side is the experimental expected reward for arm r under a uniformly randomised policy (known from prior data). The right side decomposes it into intent-conditional expectations weighted by the intent prior $P(x_k)$.

Consistency axiom: When intent equals action, counterfactual and factual outcomes coincide:

$$\mathbb{E}[Y_{x_r} \mid X = x_r] = \mathbb{E}[Y \mid X = x_r]$$

This is because $X = x_r$ implies the agent *already* chose arm r in a world where intent was x_r, so no counterfactual substitution is required. The diagonal cells of the counterfactual matrix are therefore directly observable from natural behaviour.

B.2 Cross-Intent Estimator

To isolate the entry $\mathbb{E}[Y_{x_r} \mid x_w]$, rearrange the master equation:

$$\mathbb{E}[Y_{x_r}] = \mathbb{E}[Y_{x_r} \mid x_w]\, P(x_w) + \sum_{i \neq w} \mathbb{E}[Y_{x_r} \mid x_i]\, P(x_i)$$

$$\Rightarrow \quad \mathbb{E}_{\text{XInt}}[Y_{x_r} \mid x_w] = \frac{\mathbb{E}[Y_{x_r}] - \sum_{i \neq w} \mathbb{E}[Y_{x_r} \mid x_i]\, P(x_i)}{P(x_w)}$$

This is valid whenever $P(x_w) > 0$. The unknown target cell is expressed entirely in terms of (i) the known experimental average and (ii) the remaining cells of the same column, which are populated as the agent explores.

B.3 Cross-Arm Estimator

Consider two arms x_r and x_s, both under the same intent x_w. Writing the master equation for each:

$$\mathbb{E}[Y_{x_r}] = \mathbb{E}[Y_{x_r} \mid x_w]\, P(x_w) + \sum_{i \neq w} \mathbb{E}[Y_{x_r} \mid x_i]\, P(x_i)$$

$$\mathbb{E}[Y_{x_s}] = \mathbb{E}[Y_{x_s} \mid x_w]\, P(x_w) + \sum_{i \neq w} \mathbb{E}[Y_{x_s} \mid x_i]\, P(x_i)$$

Define the *residual* for arm r under intent w as:

$$R_r^w \;=\; \mathbb{E}[Y_{x_r}] - \sum_{i \neq w} \mathbb{E}[Y_{x_r} \mid x_i]\, P(x_i)$$

so that $\mathbb{E}[Y_{x_r} \mid x_w] = R_r^w / P(x_w)$ and likewise $\mathbb{E}[Y_{x_s} \mid x_w] = R_s^w / P(x_w)$. Dividing:

$$\frac{\mathbb{E}[Y_{x_r} \mid x_w]}{\mathbb{E}[Y_{x_s} \mid x_w]} = \frac{R_r^w}{R_s^w}$$

Solving for the unknown entry $\mathbb{E}[Y_{x_r} \mid x_w]$ using the known entry $\mathbb{E}[Y_{x_s} \mid x_w]$:

$$\hat{h}(x_r, x_w, x_s) \;=\; \frac{R_r^w \cdot \mathbb{E}[Y_{x_s} \mid x_w]}{R_s^w}$$

Multiple reference arms x_s produce multiple estimates. These are combined via inverse-variance weighting to form the cross-arm composite:

$$\mathbb{E}_{\text{XArm}}[Y_{x_r} \mid x_w] = \frac{\sum_{s \neq r} \hat{h}(x_r, x_w, x_s) \,/\, \sigma^2_{x_s, x_w}}{\sum_{s \neq r} 1 \,/\, \sigma^2_{x_s, x_w}}$$

where $\sigma^2_{x_s,x_w} = \hat{p}(1-\hat{p})/N[w,s]$ is the sample variance of the Bernoulli estimate for arm s under intent w.

B.4 Combined Estimator

All three sources - direct sample, cross-intent, cross-arm - are fused via a final inverse-variance weighted average. Let α and β be defined as:

$$\alpha = \frac{\mathbb{E}_{\text{samp}}}{\sigma^2_{\text{samp}}} + \frac{\mathbb{E}_{\text{XInt}}}{\sigma^2_{\text{XInt}}} + \frac{\mathbb{E}_{\text{XArm}}}{\sigma^2_{\text{XArm}}}, \qquad \beta = \frac{1}{\sigma^2_{\text{samp}}} + \frac{1}{\sigma^2_{\text{XInt}}} + \frac{1}{\sigma^2_{\text{XArm}}}$$

$$\mathbb{E}_{\text{combo}}[Y_{x_r} \mid x_w] = \frac{\alpha}{\beta}$$

This is the standard minimum-variance unbiased combination of independent estimators: each source is trusted in inverse proportion to its variance, and the resulting pooled estimator achieves lower variance than any single source alone.

C Python Source Code

Code repository can be found at `https://github.com/AdetsGithub/Counterfactual-Data-Fusion-i`

C.1 Environment - ‘scenarios/original/greedy_casino.py‘

```
"""Greedy Casino MABUC environment."""

import numpy as np

class GreedyCasino:
    PAYOUT_MATRIX = np.array(
        [
            [0.20, 0.30, 0.50, 0.60],
            [0.60, 0.20, 0.30, 0.50],
            [0.50, 0.60, 0.20, 0.30],
            [0.30, 0.50, 0.60, 0.20],
        ],
        dtype=np.float64,
    )
```

```
    def __init__(self, rng=None):
        self.rng = rng if rng is not None else np.random.default_rng()
        self._current_intent = None

    def get_intent(self):
        b = self.rng.binomial(1, 0.5)
        d = self.rng.binomial(1, 0.5)
        self._current_intent = int(b + 2 * d)
        return self._current_intent

    def pull_arm(self, action):
        if self._current_intent is None:
            raise RuntimeError("call get_intent() before pull_arm()")
        p_win = self.PAYOUT_MATRIX[int(action), self._current_intent]
        return int(self.rng.binomial(1, p_win))
```

C.2 Baseline - ‘core/standard_mab.py‘

```
"""Naive epsilon-greedy bandit that ignores intent."""

import numpy as np

class StandardMAB:
    def __init__(self, num_arms=4, epsilon=0.1, rng=None):
        self.num_arms = num_arms
        self.epsilon = epsilon
        self.rng = rng if rng is not None else np.random.default_rng()
        self.Q_table = np.zeros(num_arms, dtype=np.float64)
        self.N_table = np.zeros(num_arms, dtype=np.int64)

    def choose_action(self):
        if self.rng.random() < self.epsilon:
            return int(self.rng.integers(self.num_arms))
        best = np.flatnonzero(self.Q_table == self.Q_table.max())
        return int(self.rng.choice(best))

    def update(self, action, reward):
        self.N_table[action] += 1
        n = self.N_table[action]
        self.Q_table[action] += (reward - self.Q_table[action]) / n
```

C.3 RDC Agent - ‘core/rdc_agent.py‘

```
"""Standard Regret Decision Criterion agent (no data fusion)."""

import numpy as np

class RDCAgent:
    def __init__(self, num_arms=4, num_intents=4, epsilon=0.1,
                 epsilon_min=0.01, epsilon_decay=0.9995, rng=None):
        self.num_arms = num_arms
        self.num_intents = num_intents
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.rng = rng if rng is not None else np.random.default_rng()
        self.Q_table = np.zeros((num_intents, num_arms), dtype=np.float64)
        self.N_table = np.zeros((num_intents, num_arms), dtype=np.int64)

    def choose_action(self, intent):
        intent = int(intent)
        if self.rng.random() < self.epsilon:
            return int(self.rng.integers(self.num_arms))
        row = self.Q_table[intent]
        best = np.flatnonzero(row == row.max())
        return int(self.rng.choice(best))

    def update(self, intent, action, reward):
        intent, action = int(intent), int(action)
        self.N_table[intent, action] += 1
        n = self.N_table[intent, action]
        self.Q_table[intent, action] += (reward - self.Q_table[intent, action]) / n
        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

C.4 Data-Fusion RDC Agent - ‘core data_fusion_rdc_agent.py‘

```
"""Data-fusion RDC agent: cross-intent, cross-arm, and combined fusion estimates."""

import numpy as np
from core.rdc_agent import RDCAgent
```

```
_MIN_VARIANCE = 1e-12

class DataFusionRDCAgent(RDCAgent):
    def __init__(self, datasets, **kwargs):
        super().__init__(**kwargs)
        self.datasets = datasets

    def get_variance(self, intent, action):
        n = self.N_table[intent, action]
        if n < 2:
            return 1.0
        p = max(0.05, min(0.95, self.Q_table[intent, action]))
        return (p * (1.0 - p)) / n

    def _arm_residual(self, query_action, query_intent):
        residual = self.datasets.experimental[query_action]
        for i in range(self.num_intents):
            if i == query_intent:
                continue
            est = (self.Q_table[i, query_action]
                   if self.N_table[i, query_action] >= 1
                   else self.datasets.experimental[query_action])
            residual -= est * self.datasets.intent_prior[i]
        return residual

    def cross_intent_estimate(self, query_action, query_intent):
        p_w = self.datasets.intent_prior[query_intent]
        if p_w <= _MIN_VARIANCE:
            return float(self.Q_table[query_intent, query_action])
        return self._arm_residual(query_action, query_intent) / p_w

    def cross_arm_estimate(self, query_action, query_intent):
        weighted_sum, weight_sum = 0.0, 0.0
        for s in range(self.num_arms):
            if s == query_action:
                continue
            denom = self._arm_residual(s, query_intent)
            if abs(denom) <= _MIN_VARIANCE:
                continue
```

```
            alt_est = (self.Q_table[query_intent, s]
                       if self.N_table[query_intent, s] >= 1
                       else self.datasets.experimental[s])
            h = self._arm_residual(query_action, query_intent) * alt_est / denom
            if not np.isfinite(h):
                continue
            var = max(self.get_variance(query_intent, s), _MIN_VARIANCE)
            weighted_sum += h / var
            weight_sum  += 1.0 / var
        if weight_sum > 0.0:
            return weighted_sum / weight_sum
        return self.cross_intent_estimate(query_action, query_intent)

    def get_fused_estimate(self, intent, action):
        e_samp  = self.Q_table[intent, action]
        v_samp  = max(self.get_variance(intent, action), _MIN_VARIANCE)
        e_xint  = self.cross_intent_estimate(action, intent)
        v_xint  = max(self._sigma2_xint(action, intent), _MIN_VARIANCE)
        e_xarm  = self.cross_arm_estimate(action, intent)
        v_xarm  = max(self._sigma2_xarm(action, intent), _MIN_VARIANCE)
        alpha = e_samp/v_samp + e_xint/v_xint + e_xarm/v_xarm
        beta  = 1.0/v_samp  + 1.0/v_xint  + 1.0/v_xarm
        return alpha / beta

    def choose_action(self, intent):
        intent = int(intent)
        if self.rng.random() < self.epsilon:
            return int(self.rng.integers(self.num_arms))
        row  = np.array([self.get_fused_estimate(intent, a)
                         for a in range(self.num_arms)])
        best = np.flatnonzero(row == row.max())
        return int(self.rng.choice(best))
```

C.5 Prior Datasets - ‘scenarios/original/mabuc_datasets.py‘

```
"""Initial observational and experimental priors for Greedy Casino MABUC."""

from dataclasses import dataclass
import numpy as np
```

```
@dataclass(frozen=True)
class MABUCDatasets:
    observational: np.ndarray  # E[Y|X]  - natural win rate per arm
    experimental:  np.ndarray  # E[Y_x]  - randomised-trial win rate per arm
    intent_prior:  np.ndarray  # P(I)    - probability of each intent profile

    @classmethod
    def greedy_casino(cls) -> "MABUCDatasets":
        return cls(
            observational=np.full(4, 0.20, dtype=np.float64),
            experimental= np.full(4, 0.40, dtype=np.float64),
            intent_prior= np.full(4, 0.25, dtype=np.float64),
        )

GREEDY_CASINO_DATASETS = MABUCDatasets.greedy_casino()
```

C.6 Simulation Runner - ‘scenarios/original/run.py‘

```
"""Run the original 4×4 Greedy Casino simulation."""

import numpy as np
from core.data_fusion_rdc_agent import DataFusionRDCAgent
from core.rdc_agent import RDCAgent
from core.standard_mab import StandardMAB
from scenarios.original.greedy_casino import GreedyCasino
from scenarios.original.mabuc_datasets import GREEDY_CASINO_DATASETS

def run_simulation(T=50_000, seed=42):
    rng = np.random.default_rng(seed)
    env = GreedyCasino(rng=rng)
    agents = {
        "Standard MAB":    StandardMAB(rng=np.random.default_rng(seed)),
        "RDC Agent":       RDCAgent(rng=np.random.default_rng(seed)),
        "Data Fusion RDC": DataFusionRDCAgent(
                               datasets=GREEDY_CASINO_DATASETS,
                               rng=np.random.default_rng(seed)),
    }
    wins = {name: 0 for name in agents}
    histories = {name: [] for name in agents}
```

```
for t in range(1, T + 1):
    intent = env.get_intent()
    reward_cache, actions = {}, {}

    for name, agent in agents.items():
        actions[name] = (agent.choose_action()
                         if name == "Standard MAB"
                         else agent.choose_action(intent))

    for name, agent in agents.items():
        action = actions[name]
        if action not in reward_cache:
            p = GreedyCasino.PAYOUT_MATRIX[action, intent]
            reward_cache[action] = int(env.rng.binomial(1, p))
        reward = reward_cache[action]
        (agent.update(action, reward)
         if name == "Standard MAB"
         else agent.update(intent, action, reward))
        wins[name] += reward
        histories[name].append(wins[name] / t)

return {name: np.array(s) for name, s in histories.items()}
```